

Exceptional Control Flow I

Outline

- Operating Systems
- Processes
- Exceptions
- Suggested reading 1.7, 8.1

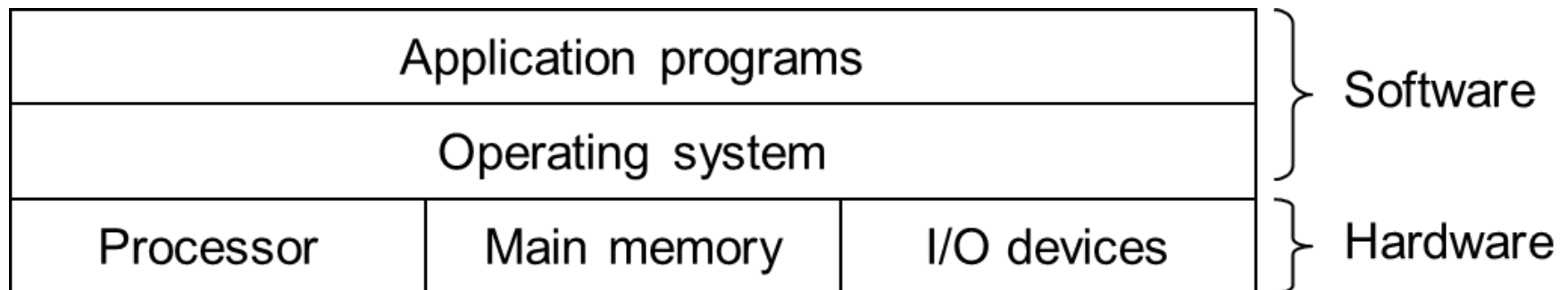
Operating Systems

Computer (Von Neumann Architecture)

- There are three components in computer hardware systems
 - Processor
 - Main memory
 - I/O devices

What Is an Operating System

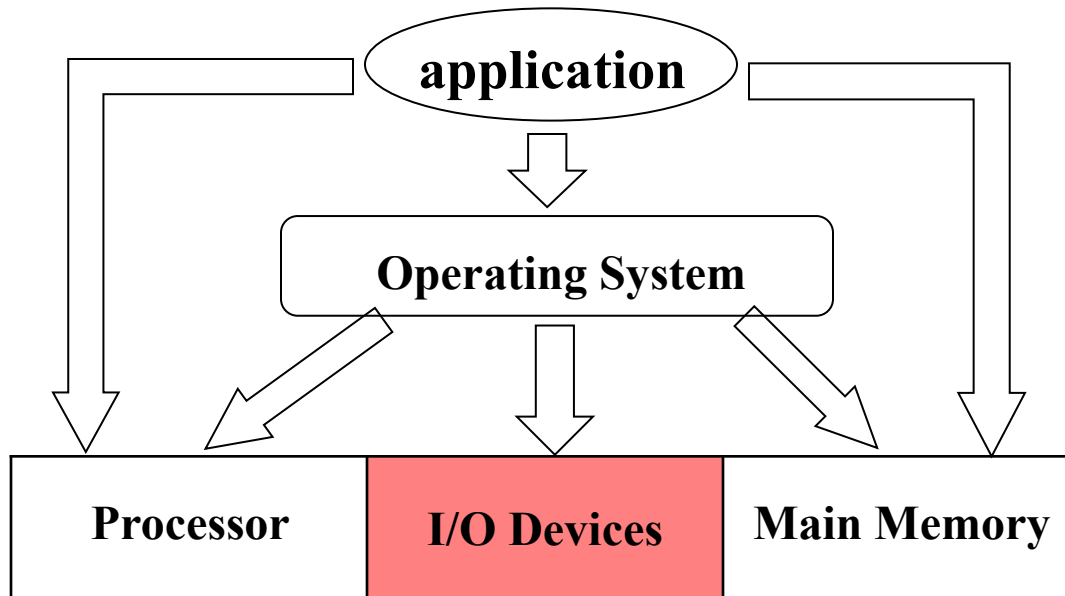
- Operating System
 - is a program that manages the computer hardware
 - also provides a basis for application programs
 - and acts as an intermediary between a user of a computer and the computer hardware



Two primary purposes

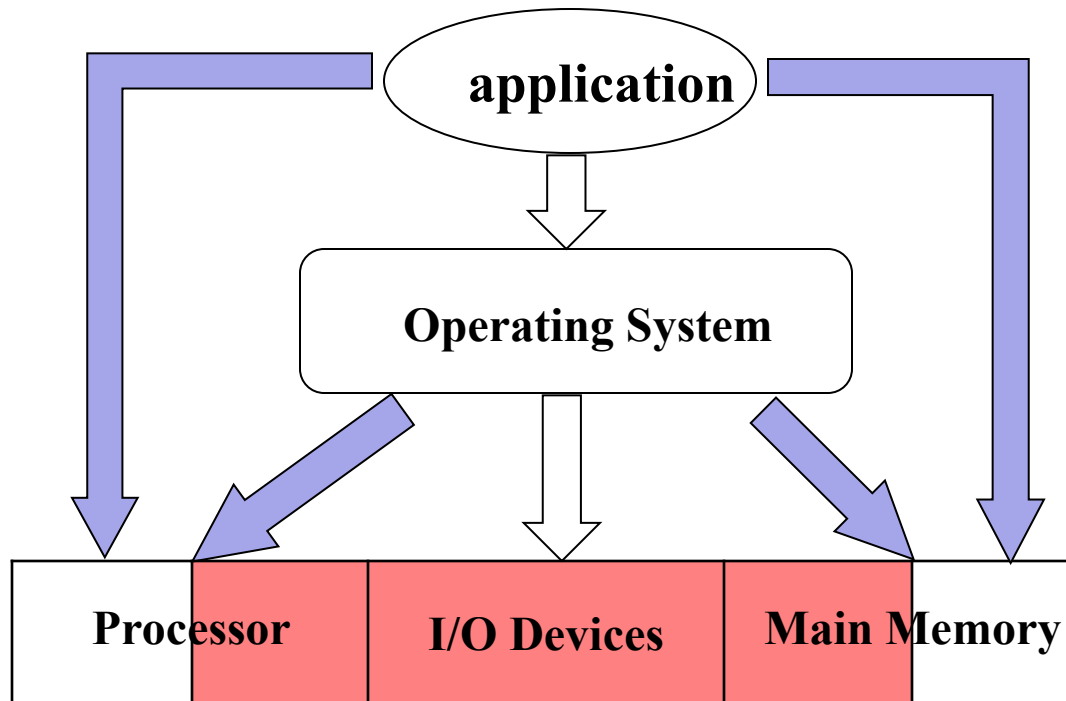
- Provide applications with simple and uniform mechanisms for manipulating hardware devices
- Protect hardware from misusing by applications

Operating Systems



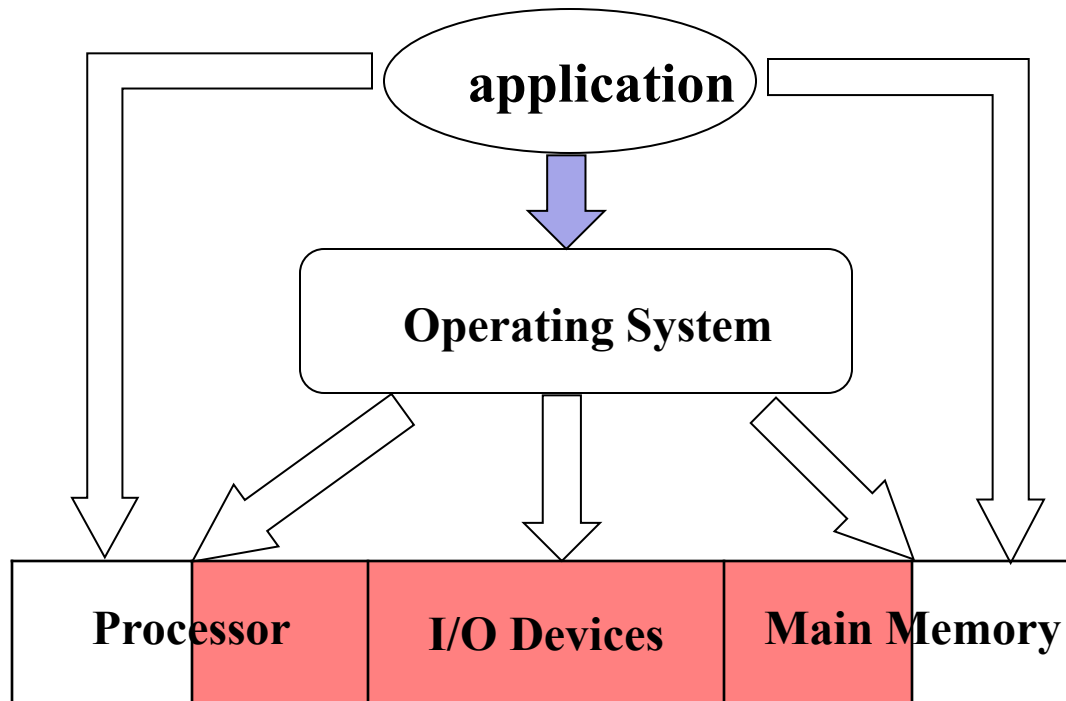
- Contemporary applications never access I/O devices directly
- They rely on the services provided by the operating system

Operating Systems



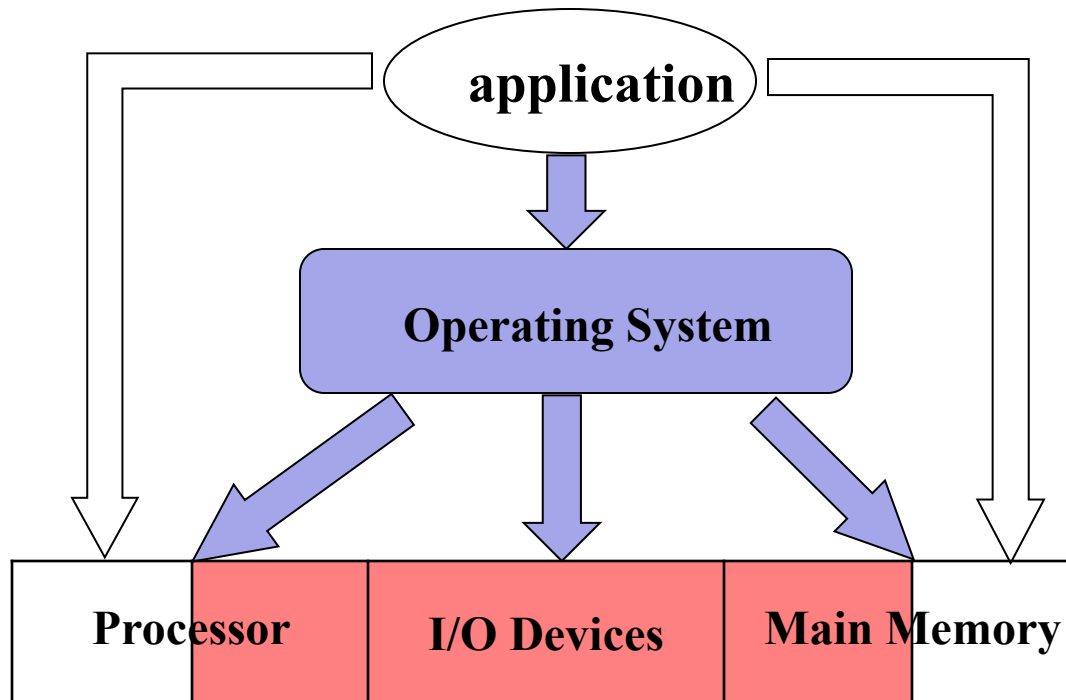
CPU and memory are assigned by OS, and directly used by applications

Operating Systems



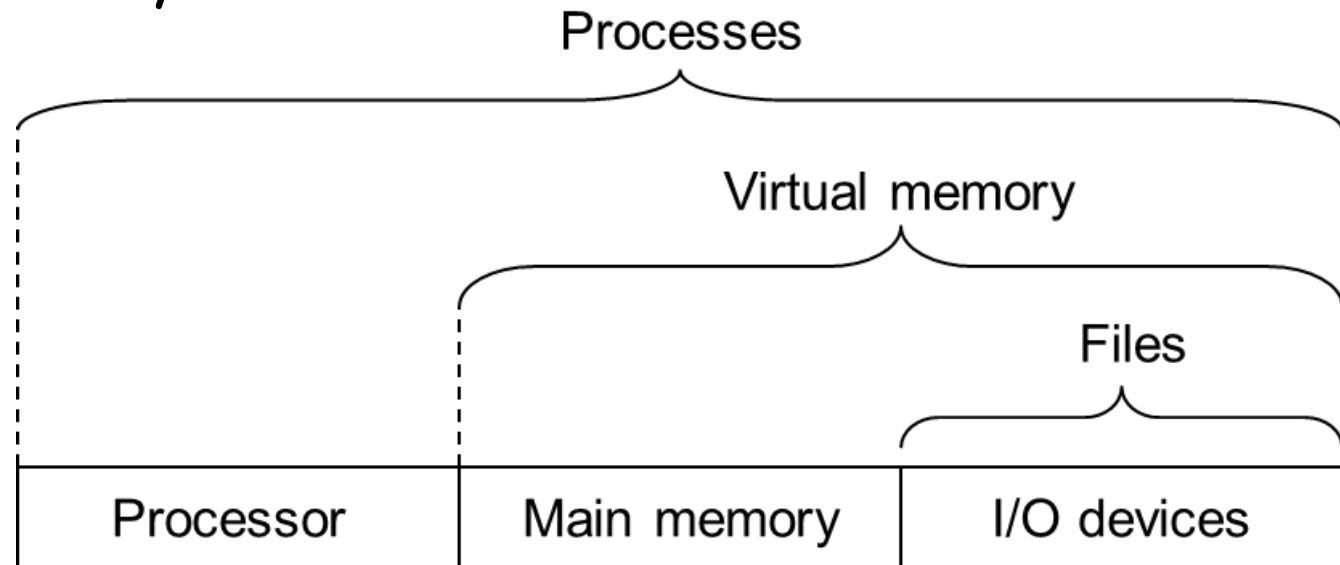
Applications rely on system calls to access hardware devices indirectly

Operating Systems



Abstractions Provided by OS

- Fundamental abstractions:
 - Process
 - Virtual memory
 - Files



Process: phenomenon

- When a program runs on a modern system
 - the operating system provides the illusion
 - that the program is the only one running on the system

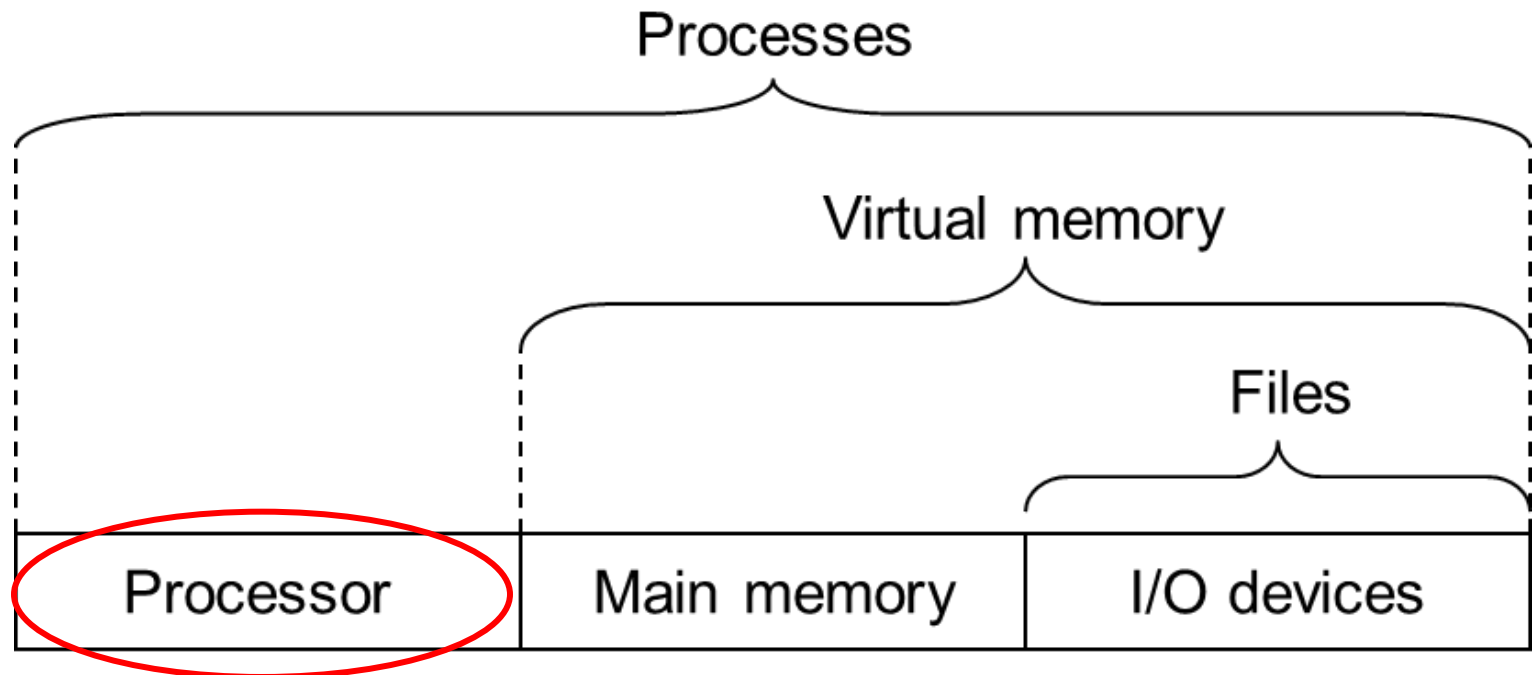
Process: phenomenon

- The program appears to have exclusive use of all the processor, main memory, and I/O devices
- The program appears to execute the instructions in the program, one after the other, without interruption
- The code and data of the program appear to be the only objects in the system's memory

Processes

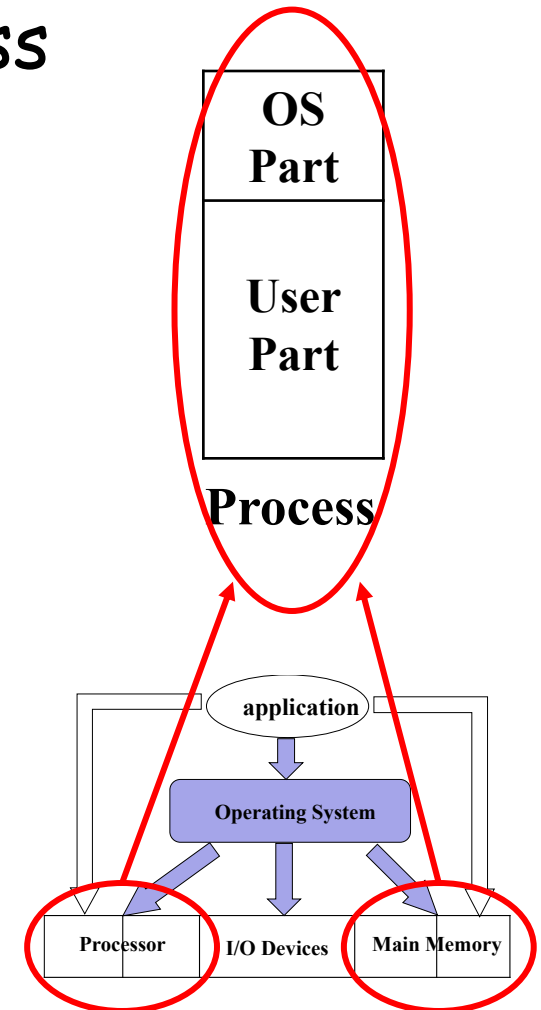
- Def: A *process* is an instance of a running program
- Process is one of the most profound ideas in computer science
- Each program in the system runs in the context of some process

Processes



Process

- There are two parts in a process
 - The user part
 - The OS part
- The OS part
 - Aka. kernel
 - Shared by all the processes



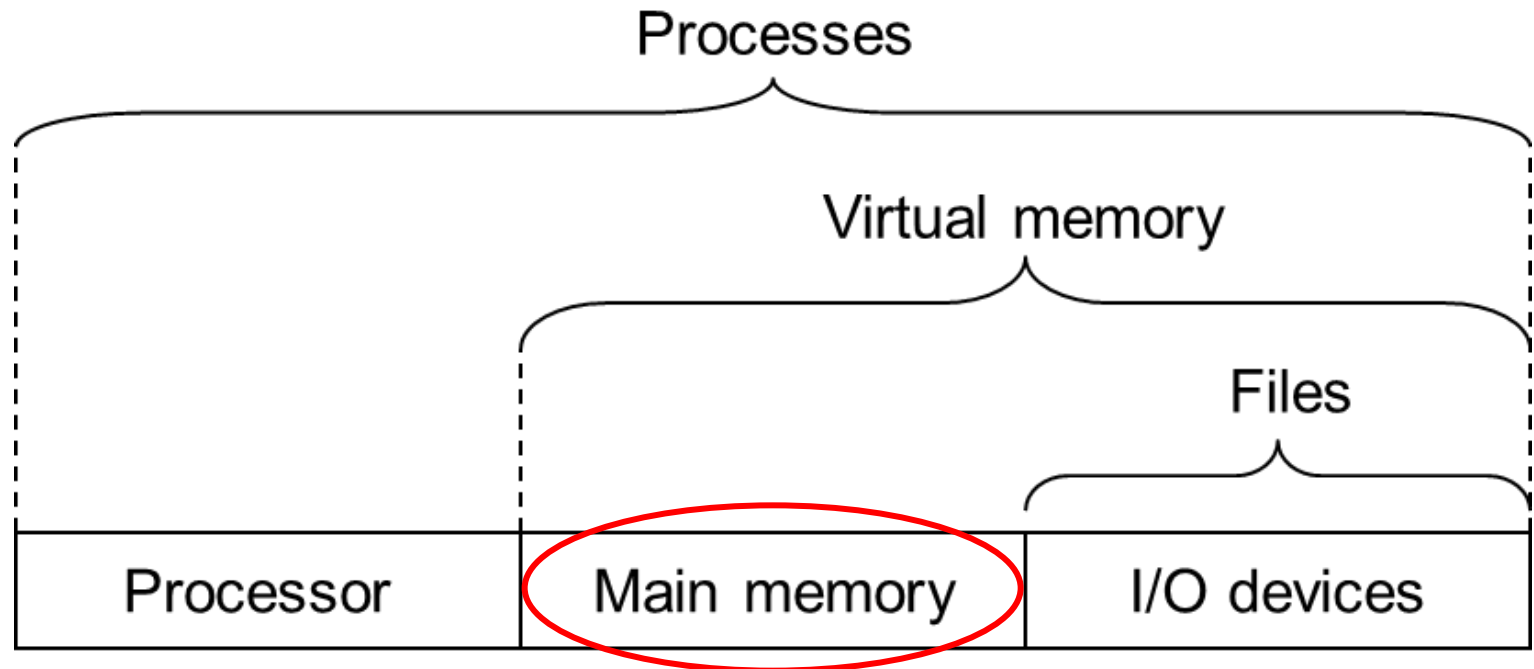
User and Kernel Modes

- Process running in **kernel mode** can
 - execute **any** instructions in the instruction set
 - access **any** memory locations in the system
- Process running in **user mode** can
 - **neither** execute privileged instructions
 - **nor** directly reference code or data in the kernel area of the address space
 - only do above indirectly via **system call** interface

Distinguish User and Kernel Modes

- There typically is **some mode bits** in some control registers
 - For example:
 - if the bit is set
 - it is in the kernel (supervisor) mode
 - if the bit is clear
 - it is in the user mode

Virtual Memory

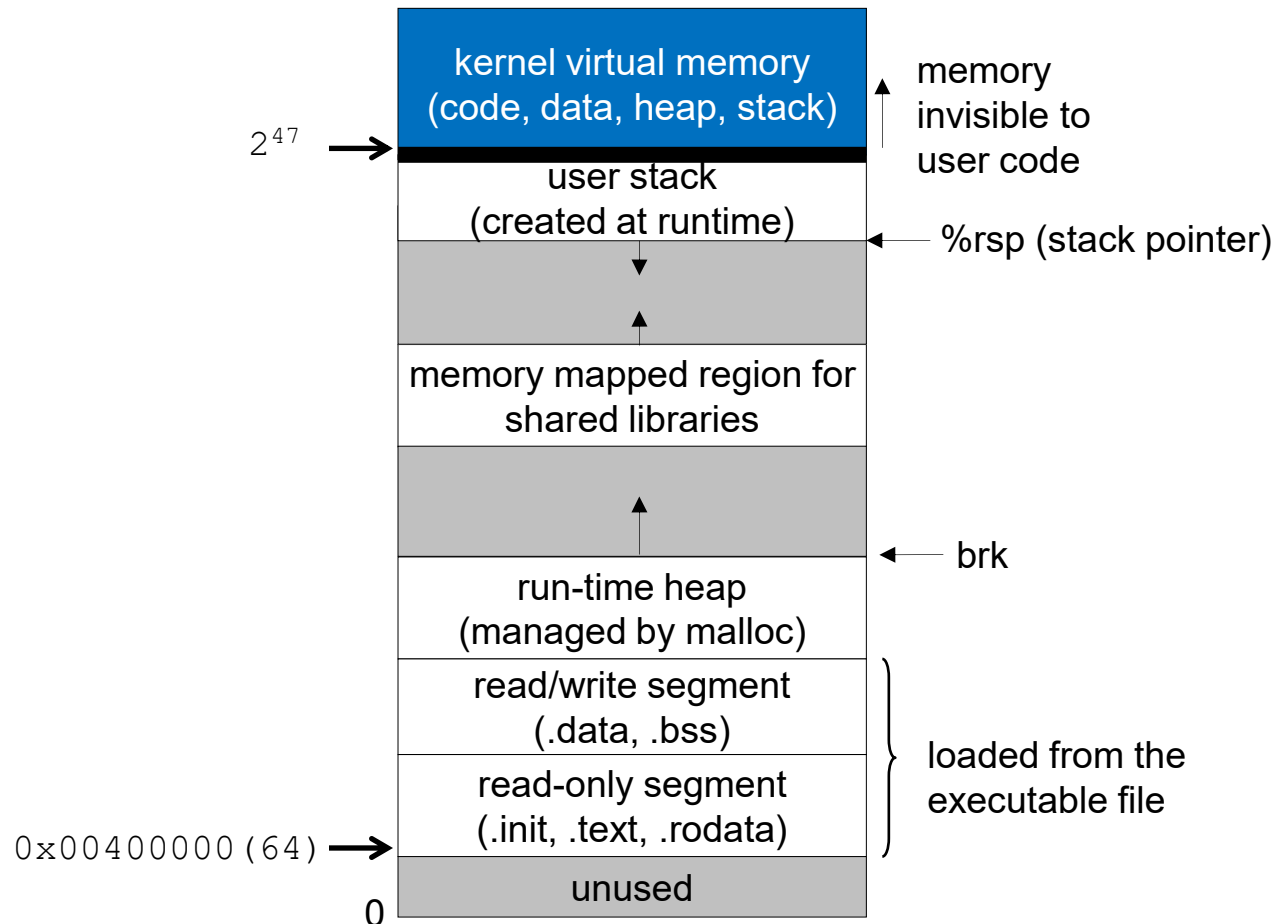


Virtual Memory

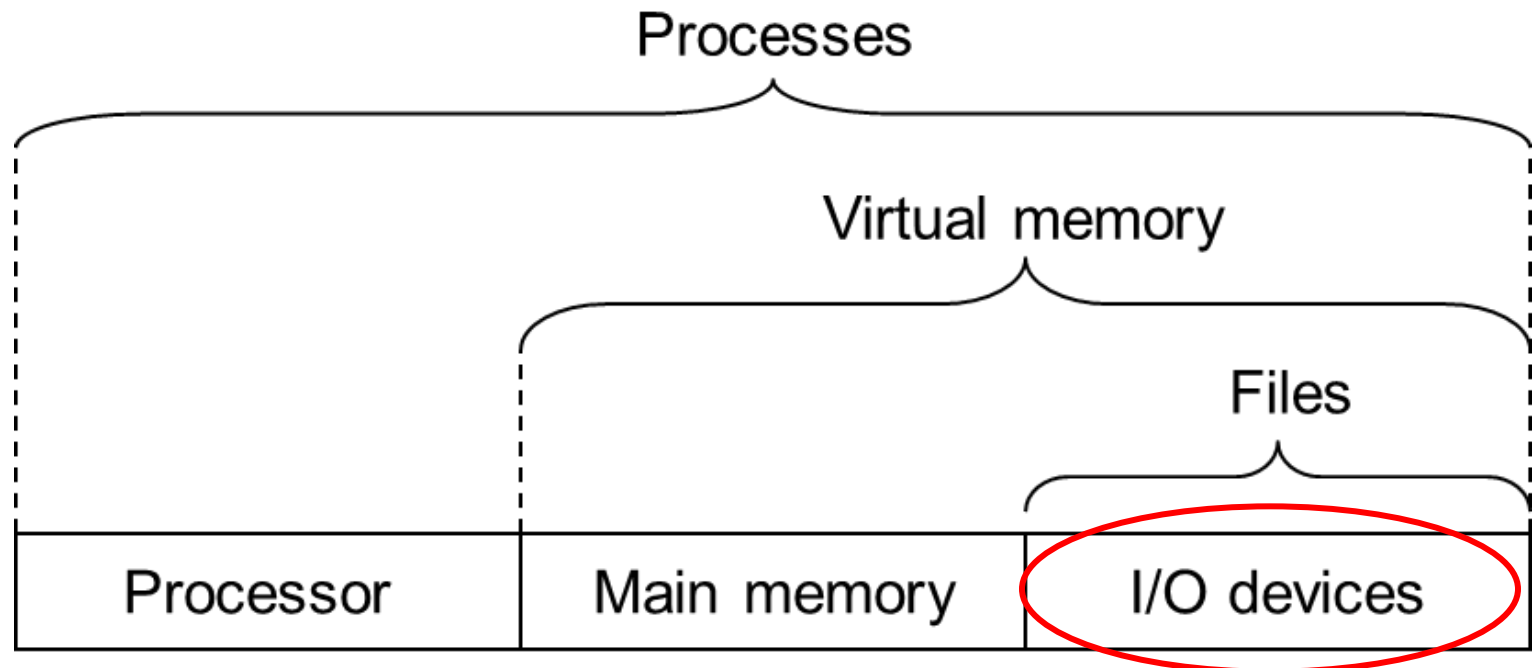
- An abstraction
 - Each process appears to have exclusive use of the main memory
 - **Virtual** address space
 - Program code and data (global variables)
 - Heap
 - Shared libraries (standard and math libraries)
 - Stack (function calls)
 - Kernel (operating system)
 - Hardware supports to translate the virtual address

Private address spaces

- Each process has its own private address space

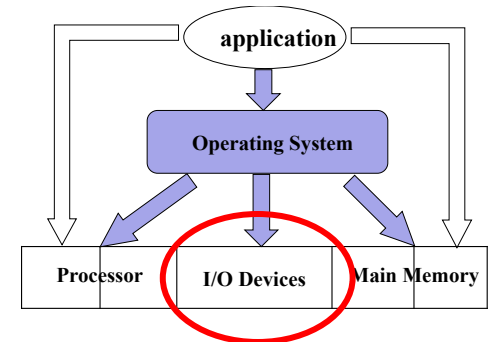


Files



Files

- Normal files: A sequence of bytes
- Each I/O device is modeled as a file
- Unix I/O
 - Using a small set of **system calls** reading and writing files
 - All input and output in the system is performed by Unix I/O



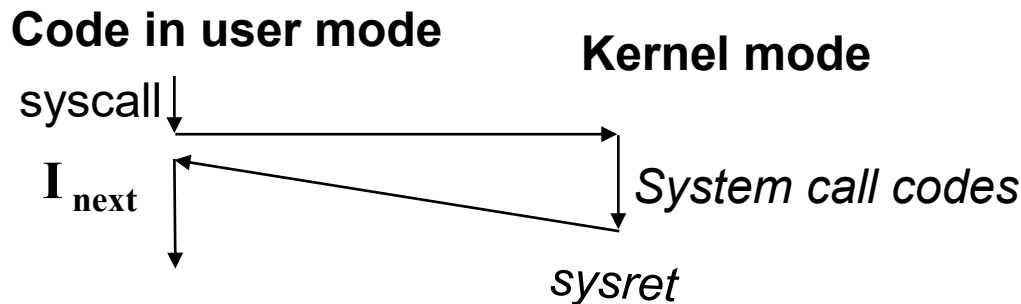
Exception

Altering the Control Flow

- A process running application code
 - initially in user mode
- How can we change the control flow from user mode to kernel mode?
 - We've discussed two mechanisms for changing the control flow:
 - jumps
 - call and return using the stack discipline
 - However, both can only alter control flow in the same mode
 - New instructions are introduced
 - **syscall / sysret**

System Calls

- The pair of instructions `syscall`/`sysret` are a hardware mechanism transfer of control between user and kernel mode
- System Calls
 - A procedure-like interface between user programs and operating systems
 - Controlled access to kernel services



System Call Example

```
# hello world
1 int main()
2 {
3     write(1, "hello, world\n", 13);
4     _exit(0);
5 }
```

Typical system calls in Linux X86-64 systems

Number	Name	Desc.	Number	Name	Desc.
0	read	Read a file	33	pause	Wait for signal
1	write	Write a file	37	alarm	Set an alarm clock
2	open	Open a file	39	getpid	Get process ID
3	close	Close a file	57	fork	Create a process
4	stat	Get file status	59	execve	Execute a program
9	mmap	Map a file into memory	60	_exit	Terminate the process
12	brk	Set the top of heap	61	wait4	Wait for process to stop
32	dup2	Duplicate a file descriptor	62	kill	Send signal to a process

System Call Example

```
1      .section .data
2      string:
3          .ascii "hello, world\n"
4      string_end:
5          .equ len, string_end - string
6      .section .text
7      .globl main
8  main:
```

System Call Example

First, call write(1, "hello, world\n", 13)

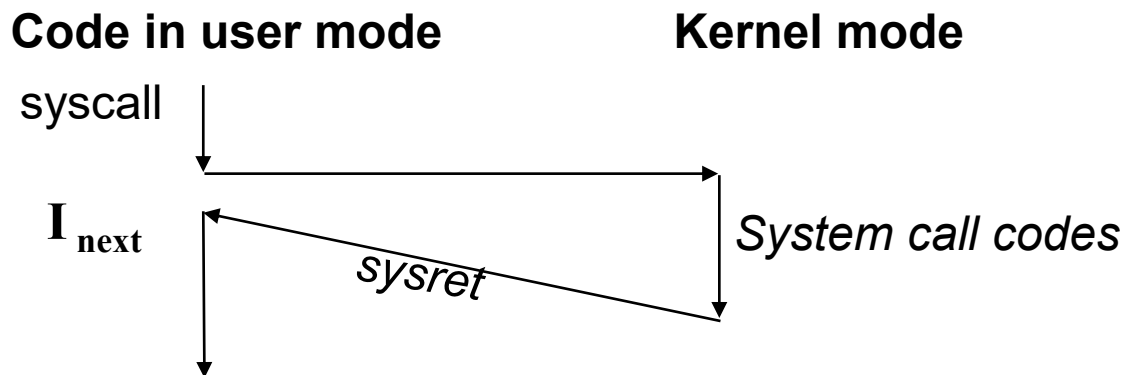
9	movq \$1, %rax	<i>write is system call 1</i>
10	movq \$1, %rdi	<i>Arg1:stdout has descriptor 1</i>
11	movq \$string, %rsi	<i>Arg2:Hello world string</i>
12	movq \$len, %rdx	<i>Arg3:string length</i>
13	syscall	<i>Make the system call</i>

Next, call exit(0)

14	movq \$60, %rax	<i>_exit is system call 60</i>
15	movq \$0, %rdi	<i>Arg1:exit status is 0</i>
16	syscall	<i>Make the system call</i>

Parameter Passing for System Calls

- Up to 6 parameters are passed between two modes
 - `%rdi,%rsi,%rdx,%r10,%r8,%r9`
- Caller saved registers need to be saved in user mode if necessary
 - `%rcx` and `%r11` are destroyed by **CPU** for saving `%rip` and `%rflags`, separately
- Returns integer in `%rax` whether it is succeeded



Q: Why we use register to pass parameter for system calls?

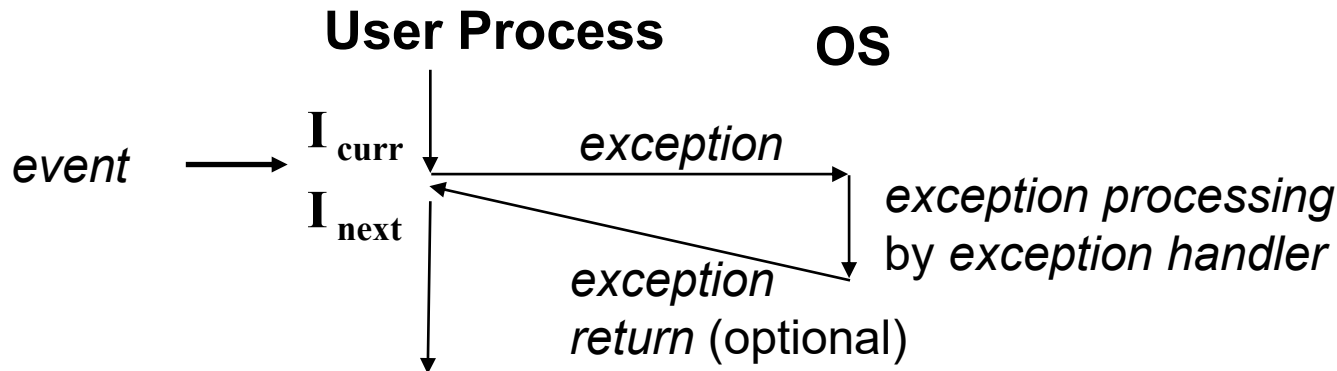
Sidebar: %rflags

- More than conditional code
- Interrupt Enable Flag (IF)
 - Whether to receive interrupts
- Trap Flag (TF)
 - Single-step execution
- ...

Event & Exception

- Processor's state
 - Encoded in various bits and signals inside the processor, such as kernel bit inside the processor
- Events
 - Significant changes in the processor 's state
 - results of execution of syscall/sysret instructions
 - data arrives from a disk or a network adapter
 - instruction doing divides by zero
- Exception
 - A hardware mechanism transfers control to the kernel in response to some **event**

Event & Exception

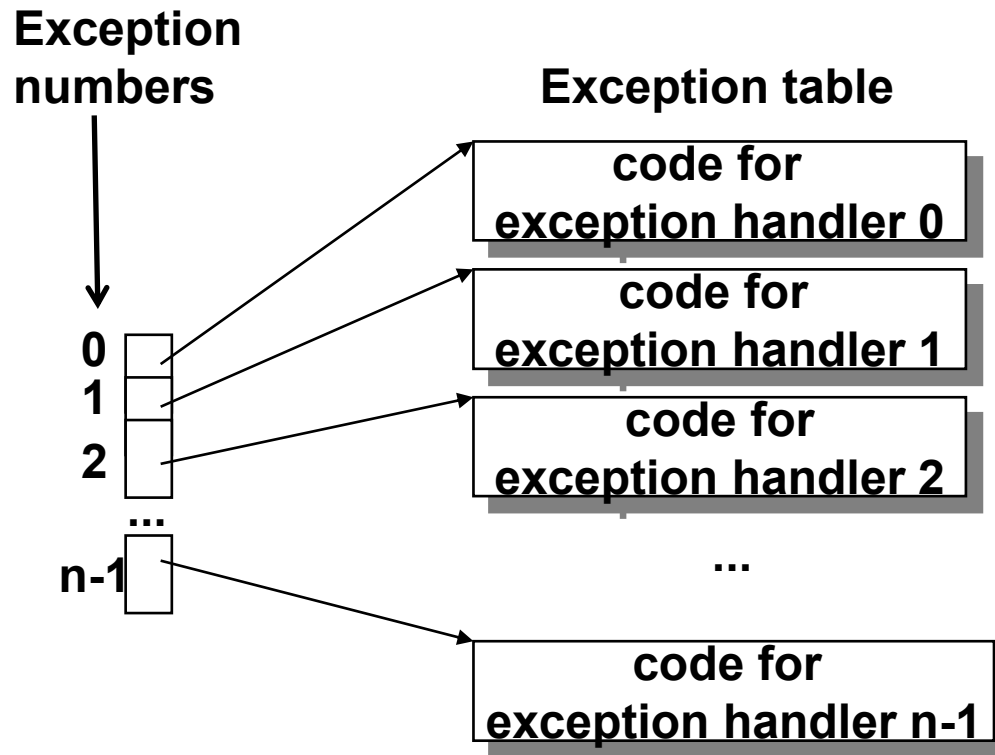


- Event may be related or unrelated to the execution of the current instruction
 - syscall, sysret (related)
 - page fault, divide zero (related)
 - a system timer goes off, an I/O request completes (unrelated)

Exception Handler

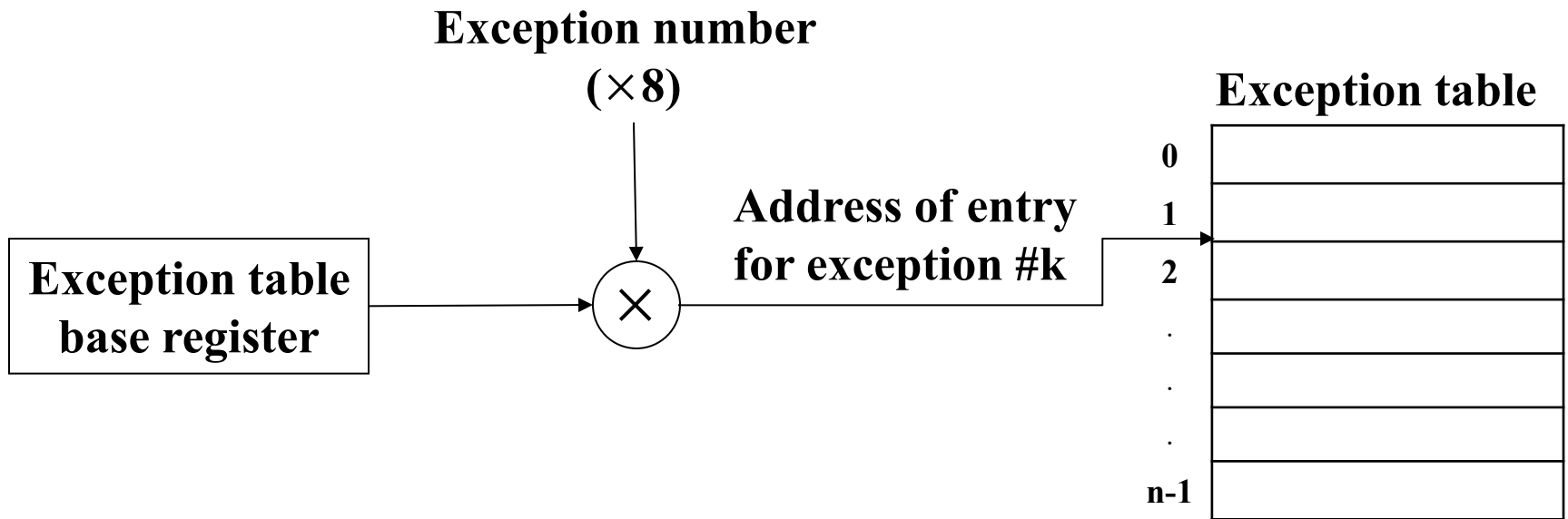
- Codes in kernel mode (OS)
- When the exception handler finishes processing, handler does one of the following:
 1. returns control to the current instruction I_{curr}
 2. returns control to the next instruction I_{next}
 3. aborts the interrupted program

Exception Table



1. Each type of event has a **unique** exception number k
2. Exception table entry k points to a function (exception handler)
3. Handler k is called each time exception k occurs

Exception Table



Exception Handler

- Exception handlers run in **kernel mode**
 - means they have complete access to all system resources
- Processor pushes necessary information onto the **kernel's** stack
 - Which are necessary to restart the interrupted program when the handler returns
 - Such as
 - Return address (either the current instruction or the next instruction)
 - RFLAGS, RSP, ...
 - Not required for syscall

Classes of Exceptions

Synchronous exceptions

- Caused by events that occur as a result of executing an instruction
 - e.g., `syscall`
- Traps
 - **Intentional**
 - returns control to "next" instruction
 - Examples: **`syscall`**, breakpoint traps

Synchronous exceptions

- Faults
 - **Unintentional** but possibly recoverable
 - Either re-executes faulting ("current") instruction or aborts.
 - Examples: **page faults** (recoverable), protection faults (unrecoverable)
- Aborts
 - **Unintentional** and unrecoverable
 - Aborts current program
 - Examples: parity error, machine check

Fault Example #1

- Memory Reference
 - User writes to memory location
 - That portion (page) of user's memory is not ready (on disk or allocation on demand)

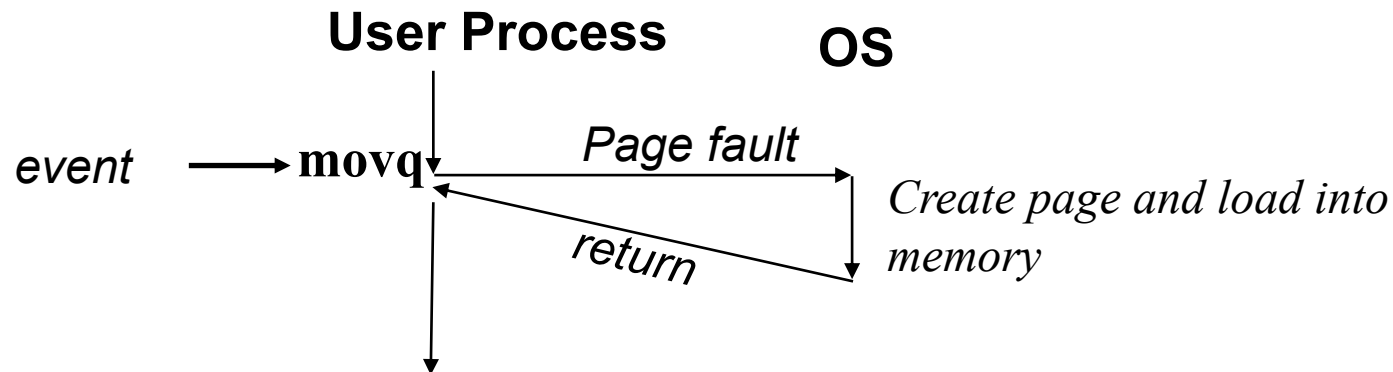
```
long a[1000];  
main ()  
{  
    a[500] = 13;  
}
```

```
4004ed: 48 c7 05 08 1b 20 00 0d 00 00 00  
        movq $0xd, 0x201b08(%rip)
```

Fault Example #1

```
int a[1000];  
main ()  
{  
    a[500] = 13;  
}
```

```
4004ed: 48 c7 05 08 1b 20 00 0d 00 00 00  
      movq $0xd, 0x201b08(%rip)
```



Fault Example #1

- Memory Reference
 - Page fault handler must load page into physical memory
 - Returns to faulting instruction
 - Successful on second try

Fault Example #2

- Memory Reference
 - User writes to memory location
 - Address is not valid

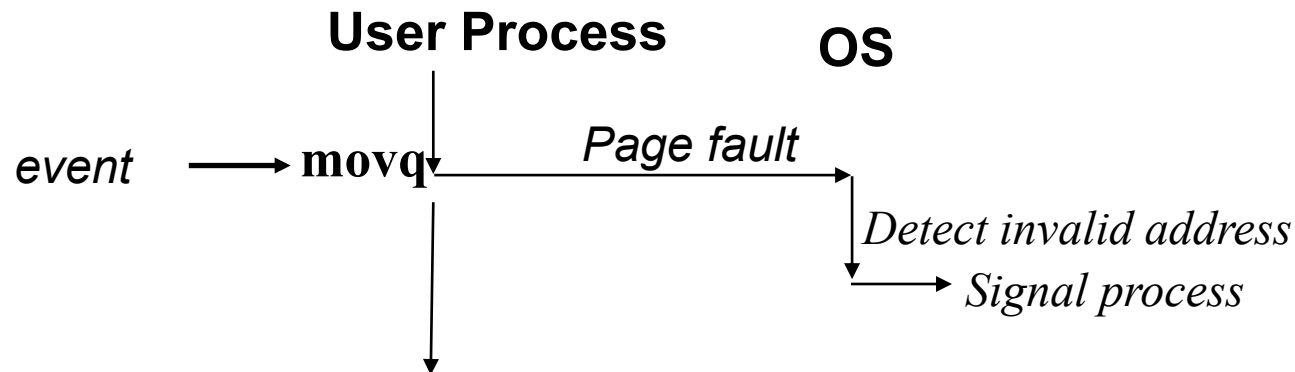
```
int a[1000];  
main ()  
{  
    a[5000] = 13;  
}
```

```
4004ed: 48 c7 05 a8 a7 20 00 0d 00 00 00  
        movq $0xd, 0x20a7a8(%rip)
```

Fault Example #2

```
int a[1000];  
main ()  
{  
    a[5000] = 13;  
}
```

```
4004ed: 48 c7 05 a8 a7 20 00 0d 00 00 00  
      movq $0xd, 0x20a7a8(%rip)
```



Fault Example #2

- Memory Reference
 - Page fault handler detects invalid address
 - Sends SIGSEGV signal to user process
 - User process exits with "segmentation fault"

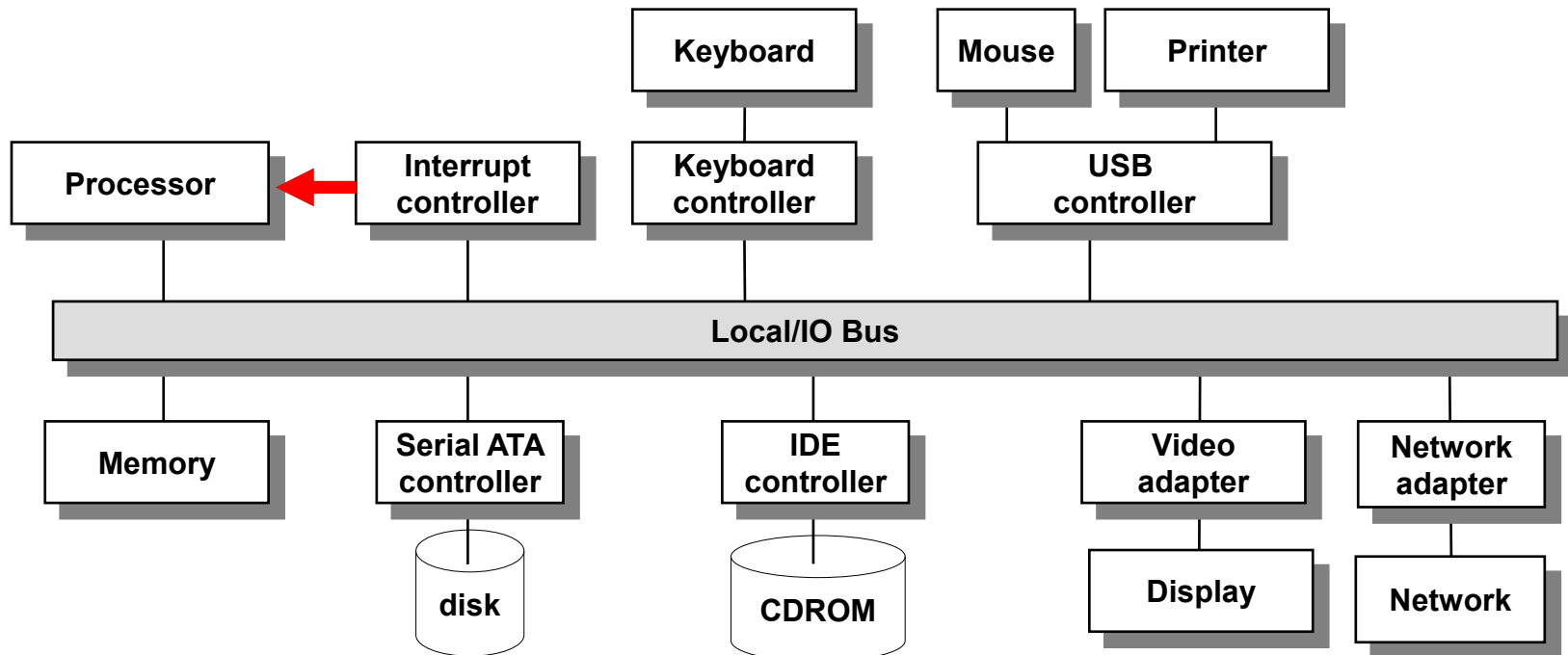
Synchronous exceptions

- Exceptions in x86-64 systems

Exception Number	Description	Exception class
0	Divide error	Fault
13	General protection fault	Fault
14	Page fault	Fault
18	Machine check	Abort
32~255	OS defined exception	Interrupt or trap

Asynchronous exceptions (interrupts)

- Caused by events **external** to the processor
 - Indicated by setting the processor's **interrupt pin**
 - Handler returns to "next" instruction.



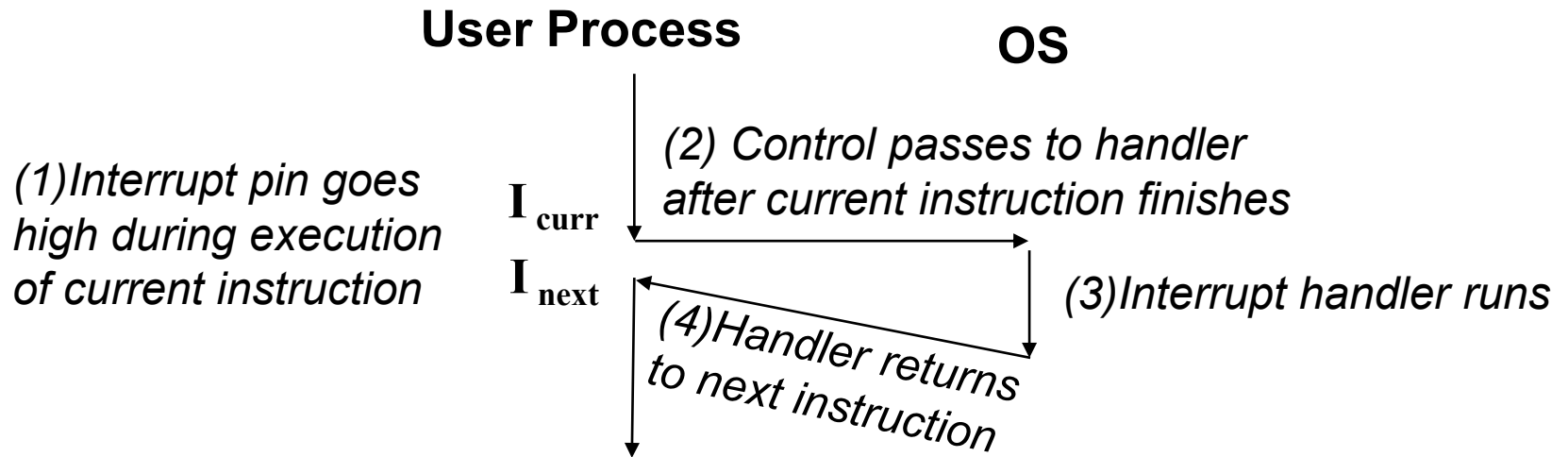
Asynchronous exceptions (interrupts)

- Examples:
 - I/O interrupts
 - hitting ctrl-c at the keyboard
 - arrival of a packet from a network
 - arrival of a data sector from a disk
 - Hard reset interrupt
 - hitting the reset button
 - Soft reset interrupt
 - hitting ctrl-alt-delete on a PC

Interrupt

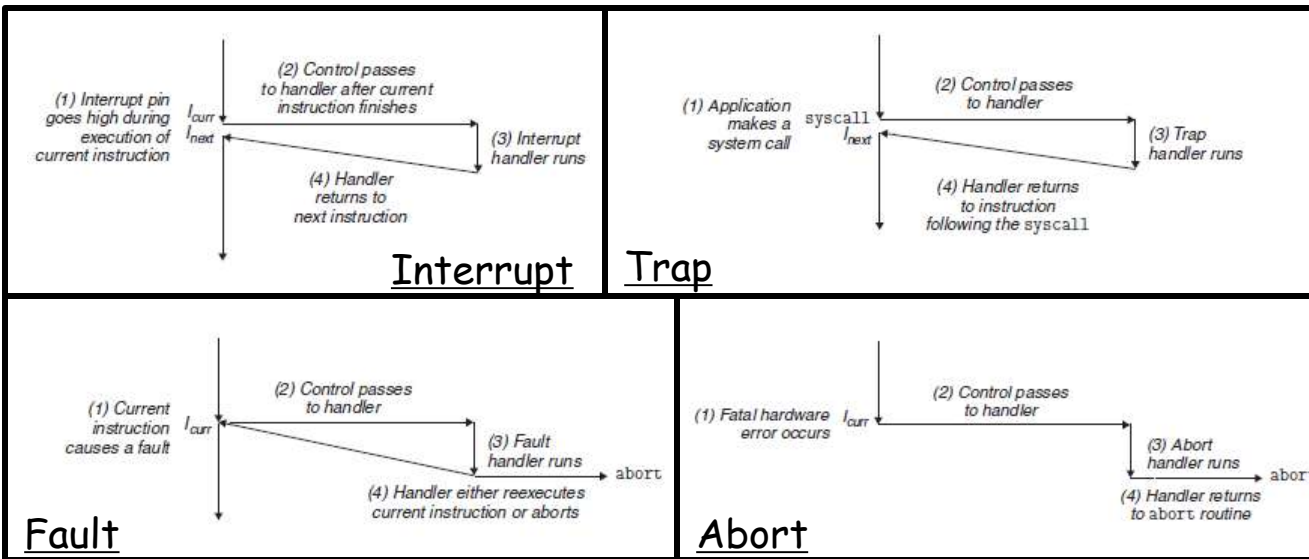
- I/O devices trigger interrupts by signaling a **pin** on the processor chip
 - network adapters, disk controllers, timer chips
- I/O devices place a number onto the system bus
 - identifies the device that caused the interrupt
- Cause the CPU stop what it is currently working on, and jump to an operating system routine (interrupt handler)

Asynchronous exceptions (interrupts)



Summary of Exceptions

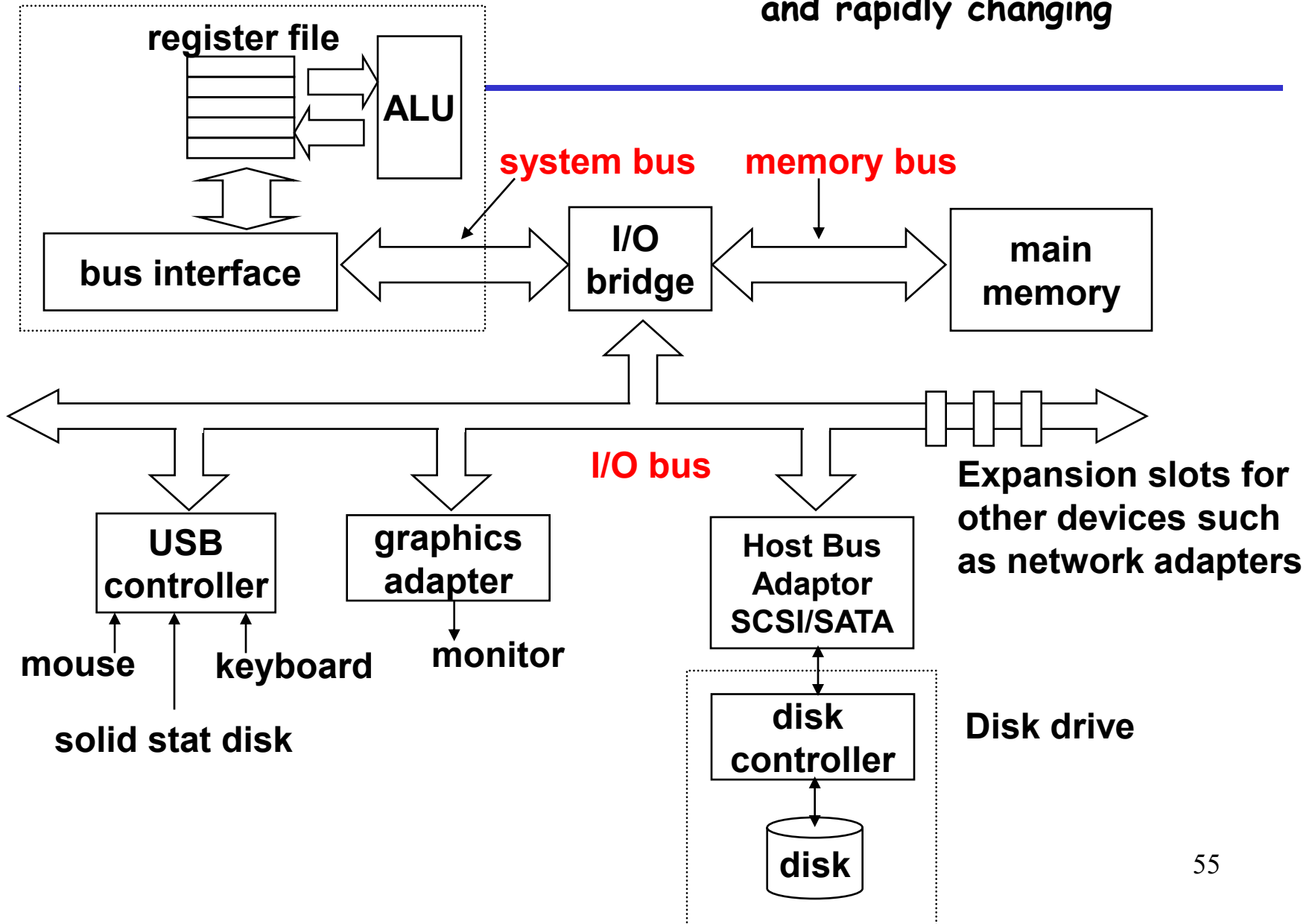
Class	Cause	Async/Sync	Return behavior
Interrupt	Signal from I/O device	Async	Always returns to next instruction
Trap	Intentional exception	Sync	Always returns to next instruction
Fault	Potentially recoverable error	Sync	Might return to current instruction
Abort	Nonrecoverable error	Sync	Never returns



A Quick Review of Disk Access

CPU chip

Tip: The bus architecture designs are various and rapidly changing

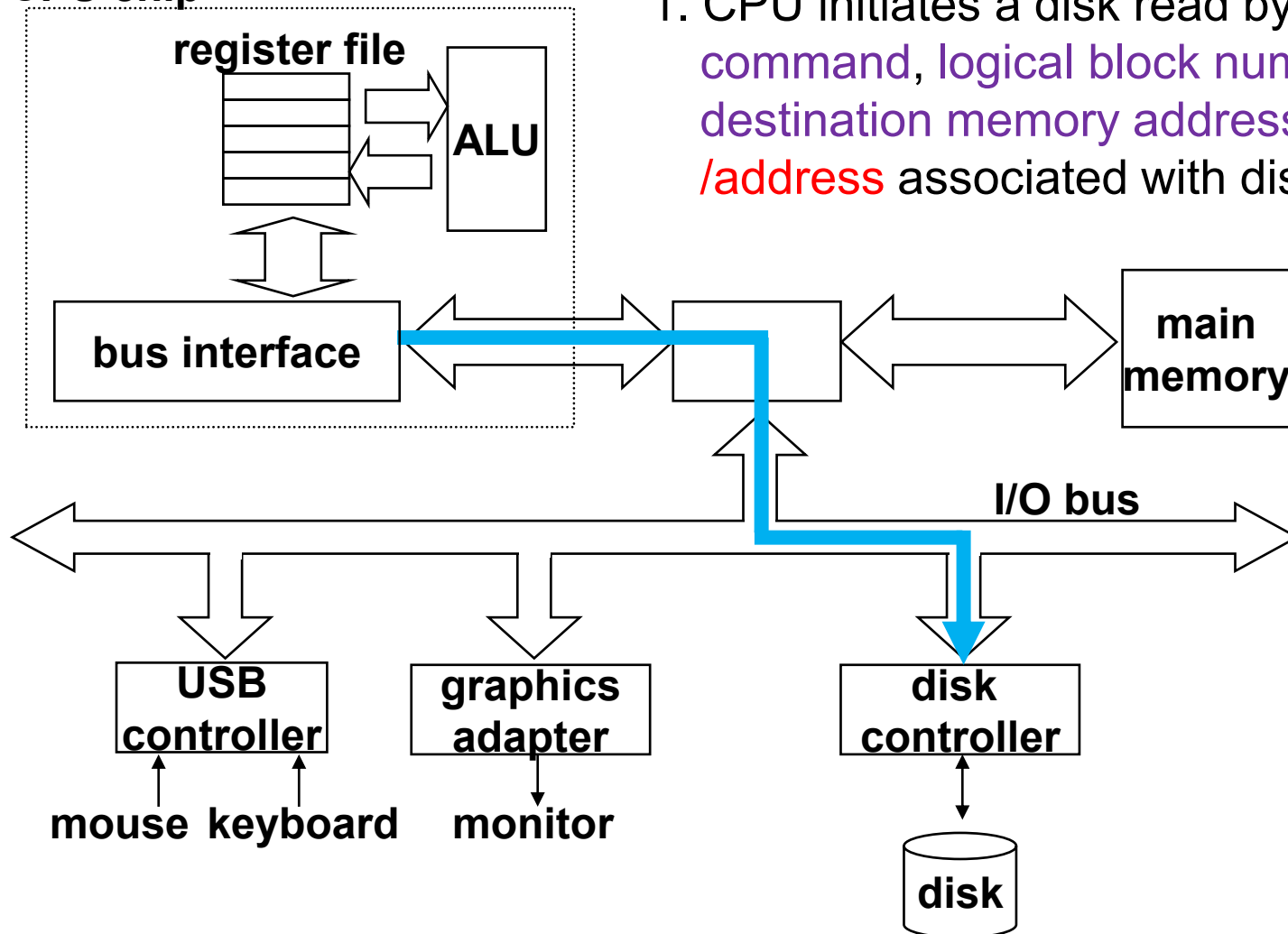


How CPU Access IO Devices

- Memory-mapped IO
 - Use **physical address** to access both DRAM or devices
 - Memory load/store instruction (**mov**)
 - Widely used
- Port I/O
 - A separate **IO address space**
 - **in/out** instruction

Reading a disk sector

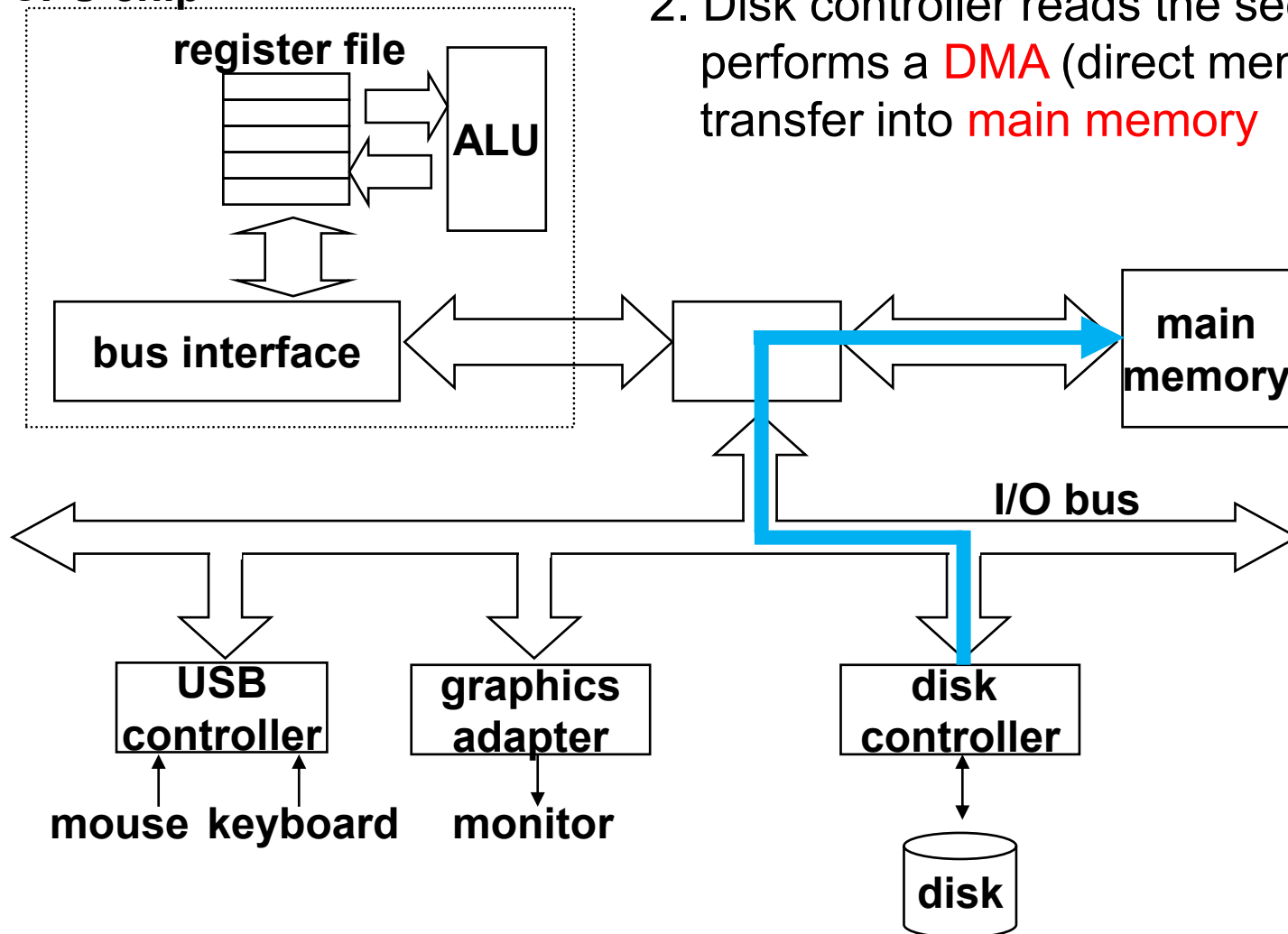
CPU chip



1. CPU initiates a disk read by writing a **command**, **logical block number**, and **destination memory address** to a **port /address** associated with disk controller

Reading a disk sector

CPU chip

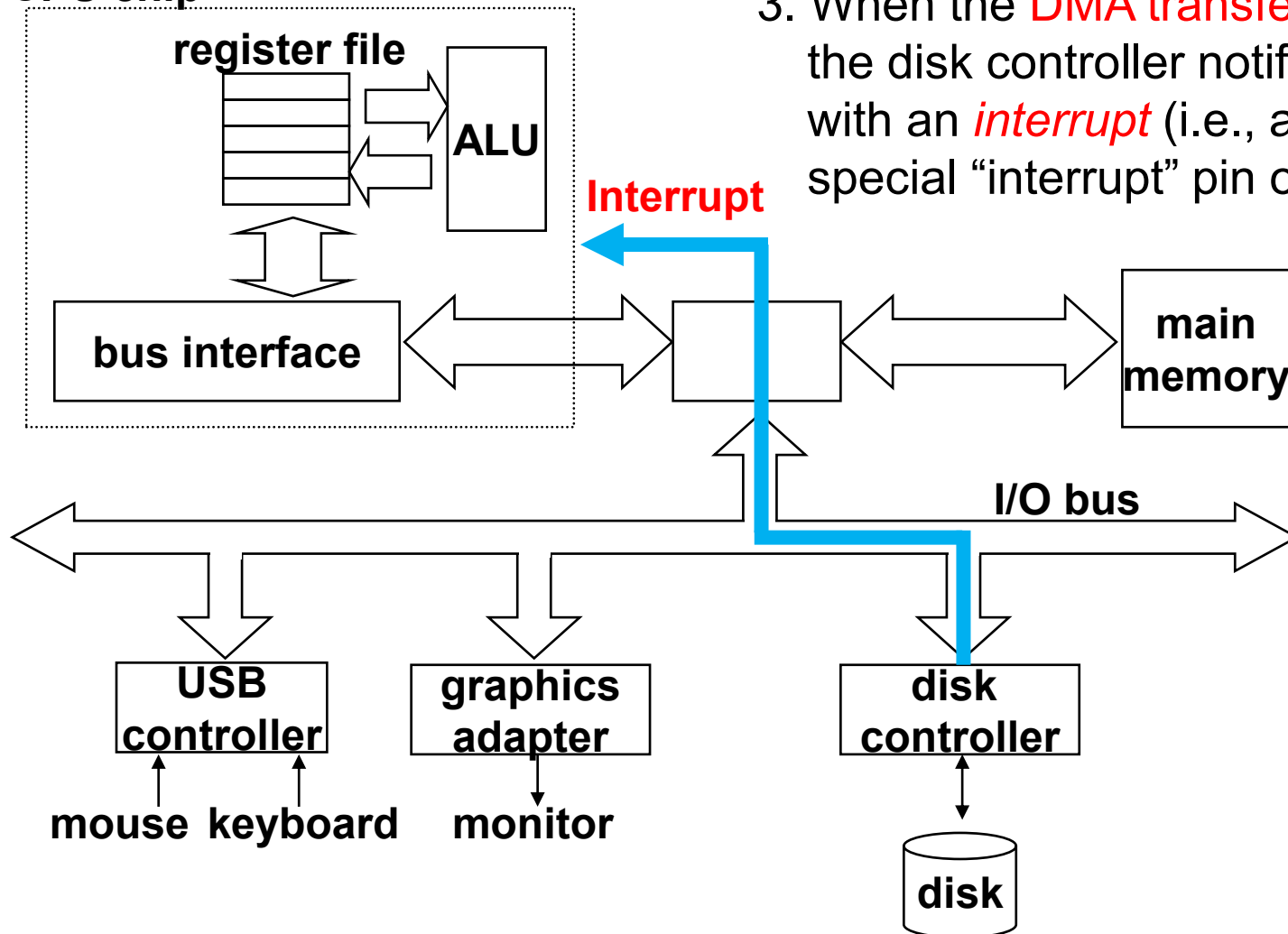


DMA

- DMA: Direct Memory Access
- A device performs a read or write bus transaction on its own, without any involvement of the CPU

Reading a disk sector

CPU chip



3. When the **DMA transfer** completes, the disk controller notifies the CPU with an **interrupt** (i.e., alerts a special “interrupt” pin on the CPU)